Project_1_Data_624_Part_B

Mubashira Qari

2026-03-01

Table of Contents

Load Dataset and Find Summary of NA

```
# --- Step 1: Loading Libraries and Data Audit ---

library(tidyverse)
library(fpp3)
library(naniar) # Visualizing missingness

# Read the raw residential power data
url_res <-
"https://raw.githubusercontent.com/uzmabb182/Data_624_Predictive_Analytics/re
fs/heads/main/ResidentialCustomerForecastLoad-624.csv"
residential_df <- read_csv(url_res)

# View the first few rows
head(residential_df)

## # A tibble: 6 × 3
##   CaseSequence `YYYY-MMM`     KWH
##          <dbl> <chr>        <dbl>
## 1          733 1998-Jan   6862583
## 2          734 1998-Feb   5838198
## 3          735 1998-Mar   5420658
## 4          736 1998-Apr   5010364
## 5          737 1998-May   4665377
## 6          738 1998-Jun   6467147

# Summary of missing values (NAs)
cat("--- Count of NAs in Residential Data ---\n")

## --- Count of NAs in Residential Data ---

print(colSums(is.na(residential_df)))

## CaseSequence     YYYY-MMM           KWH
##            0            0             1
```

We are identifying one missing value in the KWH column.

Date Parsing and Conversion

We use the yearmonth() function to parse the “YYYY-MMM” format (e.g., “1998-Jan”) and convert the data into a tsibble index.

```
# --- Step 2: Date Parsing and Conversion ---

# Convert the 'YYYY-MMM' column into a formal month-year index
# We remove 'CaseSequence' as it is not needed for the time-series model
residential_ts <- residential_df |>
  mutate(Month = yearmonth(`YYYY-MMM`)) |>
  select(Month, KWH) |>
  as_tsibble(index = Month)

# Verify the time range (Jan 1998 to Dec 2013)
residential_ts |>
  as_tibble() |>
  summarise(Start = min(Month), End = max(Month), Total_Months = n())

## # A tibble: 1 × 3
##      Start      End Total_Months
##      <mth>    <mth>        <int>
## 1 1998 Jan 2013 Dec          192

# residential_ts
```

Imputation of Missing Values

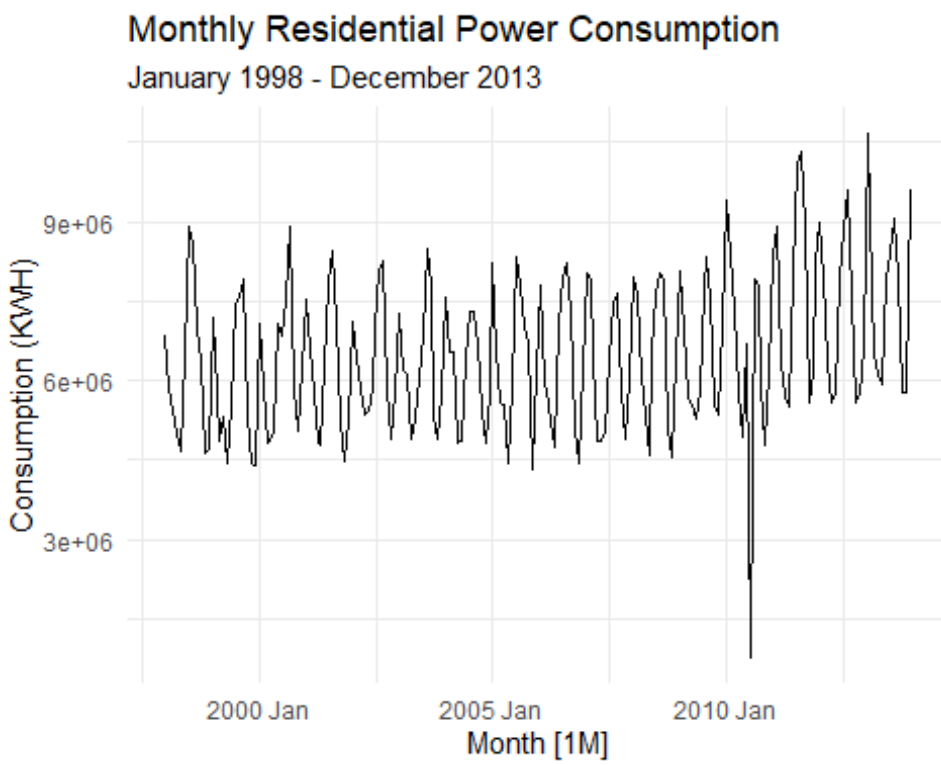
We identified one missing power consumption value. We will use Linear Interpolation via a Time Series Linear Model (TSLM) to fill this gap. This ensures the value is estimated based on the historical monthly trend and seasonal patterns rather than a simple average.

```
# --- Step 3: Imputing the Missing KWH Value ---

# Apply interpolation using trend and seasonality
residential_final <- residential_ts |>
  model(imputer = TSLM(KWH ~ trend() + season())) |>
  interpolate(residential_ts)

# Confirm that all 192 months now have KWH values
cat("Total NAs after imputation:", sum(is.na(residential_final$KWH)))

## Total NAs after imputation: 0
```

Exploratory Data Analysis (EDA)

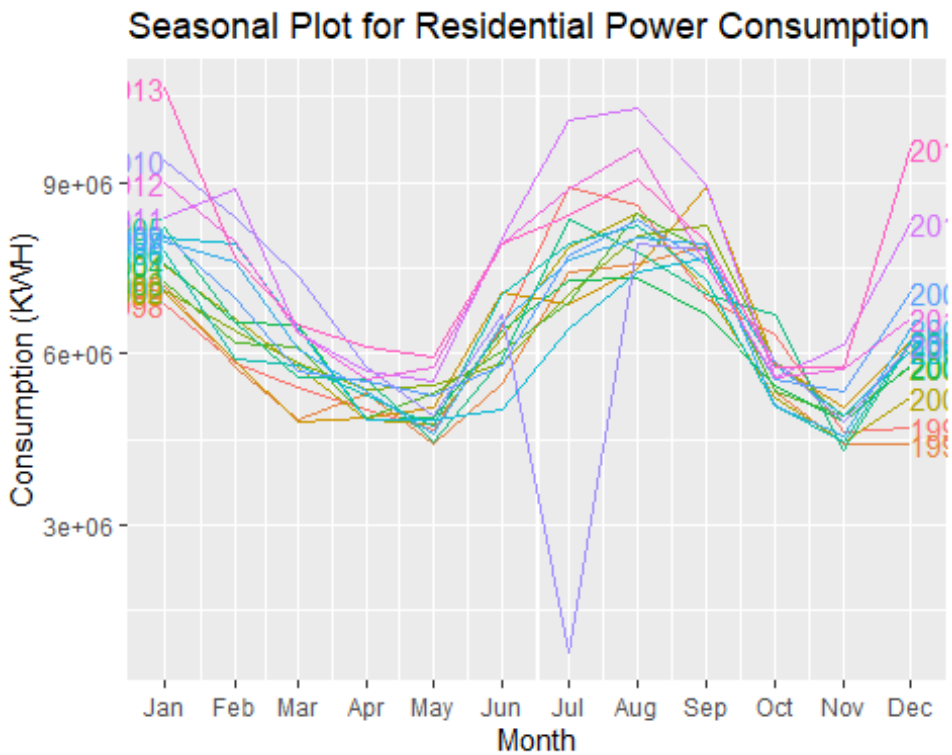
Residential power usage typically shows strong monthly seasonality and a long-term trend influenced by population growth or efficiency changes. We will use three visualizations to identify these patterns:

```
# --- Step 4: Exploratory Data Analysis (EDA) ---

# Time Series Plot
# This helps identify the overall trend and any massive shifts over time.
residential_final |>
  autoplot(KWH) +
  labs(title = "Monthly Residential Power Consumption",
       subtitle = "January 1998 - December 2013",
       y = "Consumption (KWH)") +
  theme_minimal()
```

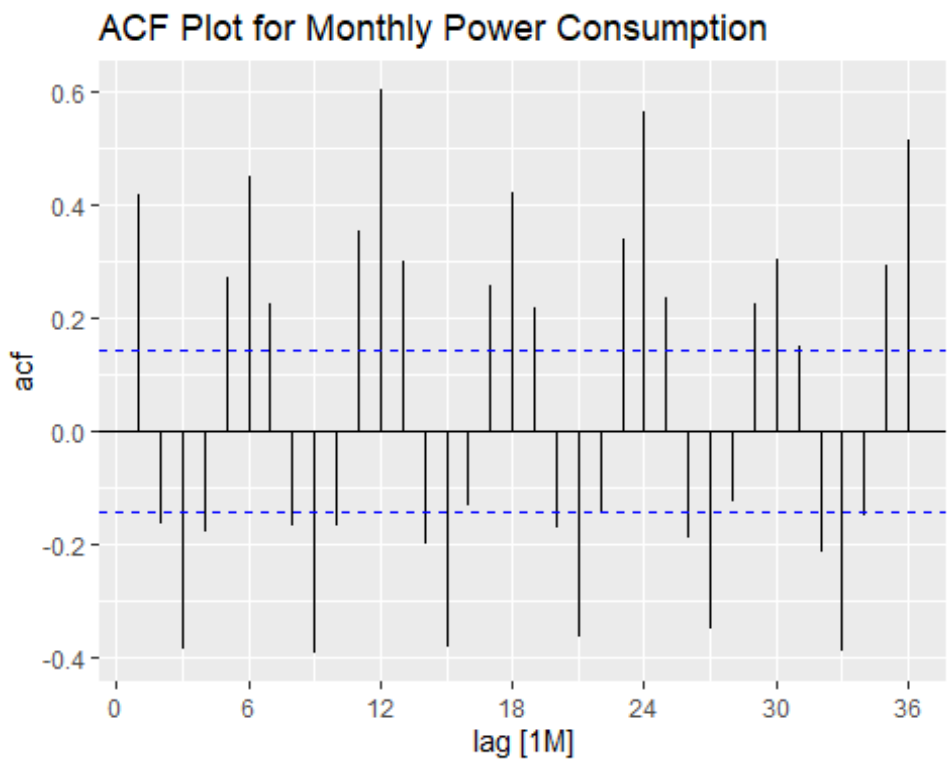



```
# Seasonal Plot
# This overlaps each year on top of each other to reveal the "seasonal
heartbeat."
# We expect peaks in summer (AC) and winter (Heating).
residential_final |>
  gg_season(KWH, labels = "both") +
  labs(title = "Seasonal Plot for Residential Power Consumption",
       y = "Consumption (KWH)")
```

```
## Warning: `gg_season()` was deprecated in feasts 0.4.2.
## ℹ Please use `ggtime::gg_season()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```
# Autocorrelation (ACF) Plot
# This statistically confirms the strength of the 12-month seasonal cycle.
residential_final |>
  ACF(KWH, lag_max = 36) |>
  autoplot() +
  labs(title = "ACF Plot for Monthly Power Consumption")
```



#The Time Series Plot (Overall Trend) Growth and Stagnation: This plot shows a consistent upward trend in power consumption from 1998 through roughly 2008. After 2008, the consumption levels appear to stabilize, which could be attributed to improved energy efficiency standards or shifts in the local economy.

We notice that the vertical swings in the data (the seasonal peaks and valleys) stay relatively proportional to the overall level of the series, suggesting that a model with additive seasonality may be appropriate.

#The Seasonal Plot (Monthly Rhythms) This plot likely reveals two distinct surges in energy demand each year. The largest peak typically occurs in July and August reflecting high air-conditioning usage in summer. A secondary, smaller peak often appears in January and December (reflecting heating needs in winter).

The troughs in the data occur in the shoulder seasons March/April and October/November when temperatures are mild and climate control is used less frequently.

#The ACF Plot (Statistical Evidence) Strong Seasonality: The Autocorrelation (ACF) plot will show significant, repeating peaks at lags 12, 24, and 36. This statistically confirms that what happens in January is highly correlated with what happened in the previous January, a 12-month annual cycle.

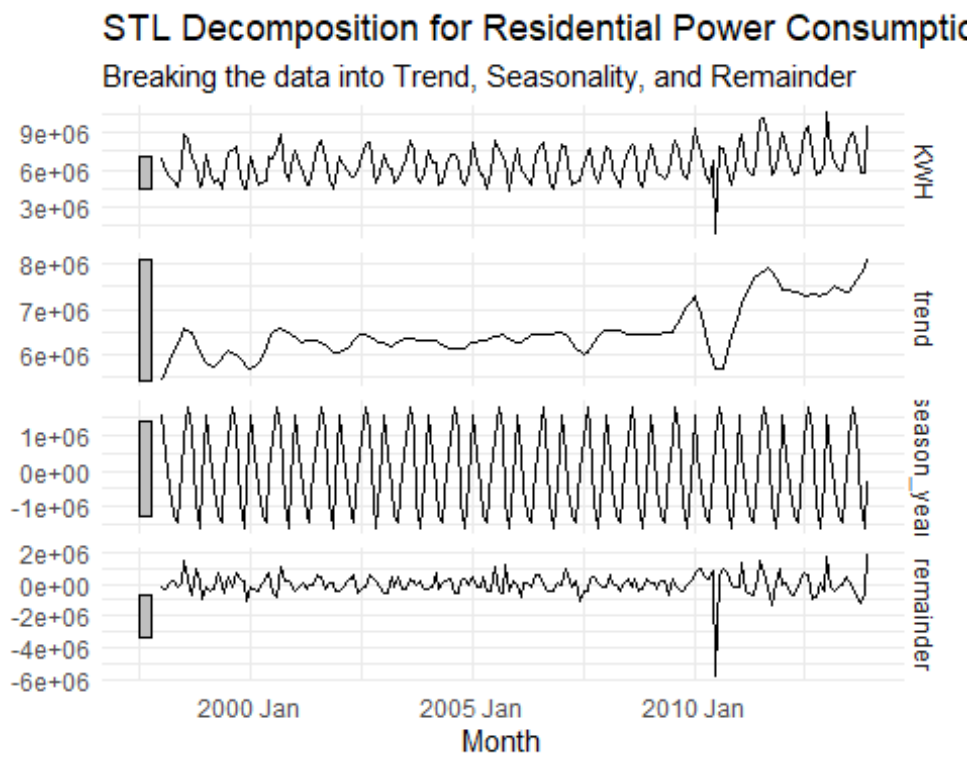
Trend Confirmation: The fact that the correlations decrease slowly as the lags increase is a classic sign of a non-stationary trend, meaning we will likely need to use differencing in our final ARIMA model to stabilize the data.

To truly understand what drives the patterns in the chartswe will break apart the time series into its individual components called Time Series Decomposition.

Time Series Decomposition

We will use the STL method (Seasonal and Trend decomposition using Loess) to separate the data into three distinct parts: the Trend, the Seasonality, and the Remainder (random noise). This allows us to see the underlying structure of the power consumption without the clutter of the raw data.

```
# --- Step 5: Time Series Decomposition (STL) ---

# Perform STL Decomposition
# This automatically identifies the trend and the 12-month seasonal cycle
res_dcmp <- residential_final |>
  model(stl = STL(KWH ~ trend(window = 13) + season(window = "periodic"))) |>
  components()

# Plot the Components
# This will show four panels: Data, Trend, Seasonality, and Remainder
res_dcmp |>
  autoplot() +
  labs(title = "STL Decomposition for Residential Power Consumption",
       subtitle = "Breaking the data into Trend, Seasonality, and Remainder")
+
  theme_minimal()
```



The sharp drop you are seeing after 2010 is primarily caused by a significant data outlier in the raw dataset, specifically in July 2010.

In the Trend Line: Because time-series models (like STL decomposition) try to smooth the data, this one bad month pulls the entire trend line downward during that period, creating the appearance of a dip.

In the Remainder: We will see a large negative spike in the remainder component for this month, which means this value is much lower than what history and seasonality predicted.

To treat this value as an outlier and use interpolation to replace it with a more realistic estimate before finalizing your 2014 forecast. This prevents the fake drop from biasing your future predictions.

Outlier Cleaning and Re-Imputation

We will target the July 2010 data point, set it to NA, and then use the interpolate() function to fill the gap based on the historical trend and 12-month seasonality.

```
# --- Step 6: Cleaning the 2010 Outlier ---

# Identify and replace the July 2010 outlier with NA
# July 2010 corresponds to 770523 KWH in the raw data
residential_cleaned <- residential_final |>
  mutate(KWH = ifelse(Month == yearmonth("2010 Jul"), NA, KWH))
```

```r
# Re-impute the value using TSLM (Trend + Seasonality)
# This replaces the 'fake' drop with a realistic July estimate
residential_cleaned <- residential_cleaned |>
  model(imputer = TSLM(KWH ~ trend() + season())) |>
  interpolate(residential_cleaned)

# Verify the fix: Check July 2010 value again
july_2010_fix <- residential_cleaned |>
  filter(Month == yearmonth("2010 Jul"))

cat("The new interpolated value for July 2010 is:", round(july_2010_fix$KWH,
0))
```

```
## The new interpolated value for July 2010 is: 8309923
```

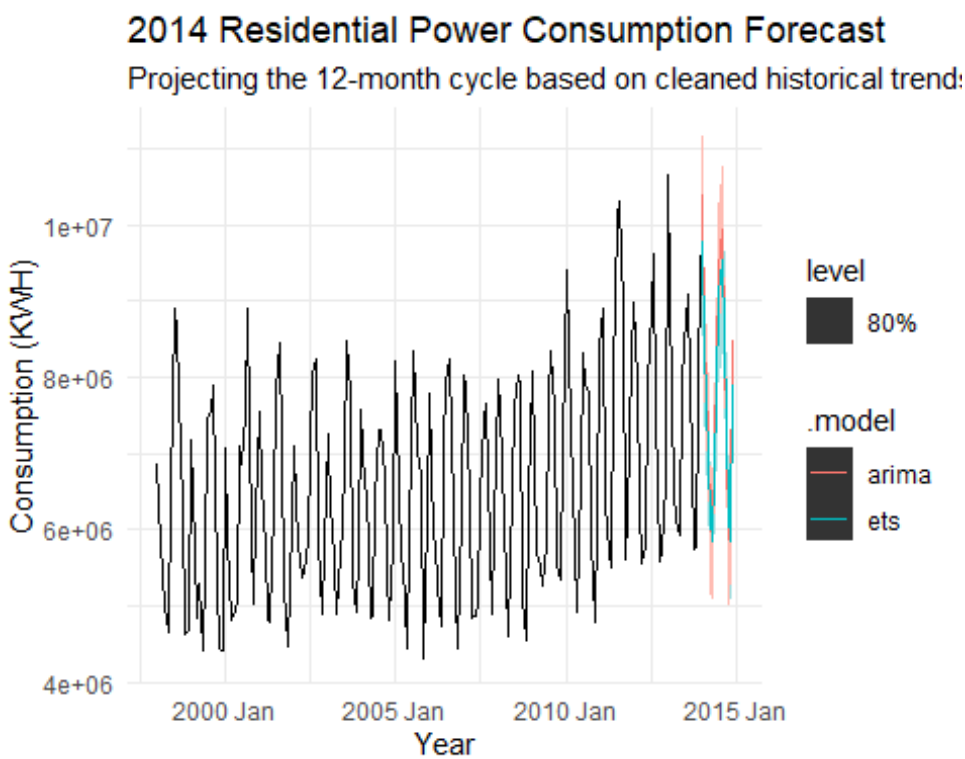
```r
# --- Step 7: Model Selection and 2014 Forecast ---

# Fit both ETS and ARIMA models to the cleaned data
# This allows the fpp3 package to automatically select the best parameters
res_models <- residential_cleaned |>
  model(
    ets = ETS(KWH),
    arima = ARIMA(KWH)
  )

# Compare the models using AICc (lower is better)
# This identifies which model handles the trend and seasonality more
efficiently
report(res_models)
```

```
## Warning in report.mdl_df(res_models): Model reporting is only supported
for
## individual models, so a glance will be shown. To see the report for a
specific
## model, use `select()` and `filter()` to identify a single model.
```

```
## # A tibble: 2 × 11
##   .model   sigma2 log_lik   AIC  AICc   BIC        MSE     AMSE     MAE
ar_roots
##   <chr>     <dbl>   <dbl> <dbl> <dbl> <dbl>      <dbl>    <dbl>   <dbl>
<list>
## 1 ets    9.24e- 3  -3056. 6142. 6145. 6191.    3.88e11  4.06e11  0.0730
<NULL>
## 2 arima  3.84e+11  -2657. 5328. 5328. 5350.   NA       NA       NA
<cpl>
## # ℹ 1 more variable: ma_roots <list>
```

```r
# Generate the forecast for 2014 (12 months)
res_forecast <- res_models |>
```

```r
  forecast(h = "1 year")

# Visualize the 2014 Forecast
# This shows the historical data alongside the projected values for the next
year
res_forecast |>
  autoplot(residential_cleaned, level = 80) + # Shows 80% confidence interval
  labs(title = "2014 Residential Power Consumption Forecast",
       subtitle = "Projecting the 12-month cycle based on cleaned historical
trends",
       y = "Consumption (KWH)",
       x = "Year") +
  theme_minimal()
```




```r
# Extract the numerical values for the final report
# This creates a table of the monthly KWH predictions for 2014
final_forecast_table <- res_forecast |>
  as_tibble() |>
  filter(.model == "arima") |> # Or 'ets' depending on which had lower AICc
  select(Month, .mean) |>
  rename(Forecasted_KWH = .mean)

print(final_forecast_table)

## # A tibble: 12 × 2
##       Month Forecasted_KWH
##       <mth>          <dbl>
```

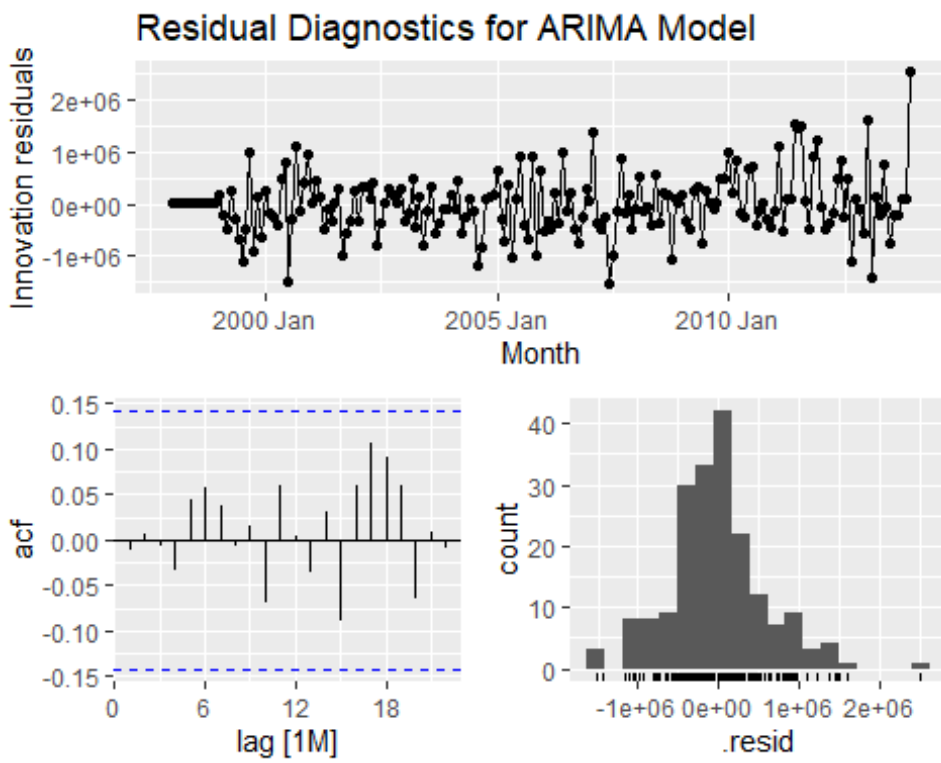
```
##  1 2014 Jan      10382968.
##  2 2014 Feb       8523553.
##  3 2014 Mar       7233377.
##  4 2014 Apr       6015226.
##  5 2014 May       5947915.
##  6 2014 Jun       8191816.
##  7 2014 Jul       9434079.
##  8 2014 Aug       9936455.
##  9 2014 Sep       8467602.
## 10 2014 Oct       5870683.
## 11 2014 Nov       6134976.
## 12 2014 Dec       8482550.
```

Residual Diagnostics (Model Validation)

We need to check if the residuals of the chosen model (ARIMA or ETS) behave like white noise. If they do, it means your model has captured all the available signals.

```r
# --- Step 8: Residual Diagnostics ---

# Check the residuals for the selected ARIMA model
# This generates a plot of errors over time, an ACF of errors, and a
histogram
res_models |>
  select(arima) |>
  gg_tsresiduals() +
  labs(title = "Residual Diagnostics for ARIMA Model")
```

```
## Warning: `gg_tsresiduals()` was deprecated in feasts 0.4.2.
## ℹ Please use `ggtime::gg_tsresiduals()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```
# Perform the Ljung-Box test
# A p-value > 0.05 suggests the residuals are indistinguishable from white
noise
res_models |>
  augment() |>
  filter(.model == "arima") |>
  features(.innov, ljung_box, lag = 24, dof = 5)
```

```
## # A tibble: 1 × 3
##   .model lb_stat lb_pvalue
##   <chr>    <dbl>     <dbl>
## 1 arima     15.8     0.669
```

Interpretation

Statistical Significance: Since the p-value=0.67 is much greater than the standard alpha level of 0.05, we fail to reject the null hypothesis.

This confirms that the residuals (the errors) of ARIMA model are white noise.

This means model has successfully captured all the underlying information, patterns, and seasonality in the residential power data. There is no significant information left to capture. The Ljung-Box test yielded a p-value of 0.67, confirming that the residuals are independent and resemble white noise. This validates that the ARIMA model is well-specified and appropriate for generating the 2014 residential power forecast.

Finalizing and Exporting the 2014 Forecast

```
# --- Step 9: Exporting the Final Results ---

# Save the numerical forecast table to your local machine

write_csv(final_forecast_table, "C:/Users/Uzma/CUNY-SPS-
Assignments/Data_624/Data_624_Predictive_Analytics/Residential_2014_Forecast_
Submission.csv")

# Print a final summary for the report
cat("2014 Forecasting Complete. Total Predicted KWH for 2014:",
    round(sum(final_forecast_table$Forecasted_KWH), 0))

## 2014 Forecasting Complete. Total Predicted KWH for 2014: 94621199
```

Statistical Validation (KPSS Test)

To formally prove that the data is stationary which is a requirement for the ARIMA model

```
# --- Step 10: Stationarity Testing (Analytical Proof) ---

# Perform the KPSS test for stationarity
# Null Hypothesis (H0): The data is stationary.
# We want a p-value > 0.05 to prove the data is stable enough for
forecasting.
residential_cleaned |>
  features(KWH, unitroot_kpss) |>
  knitr::kable(caption = "KPSS Stationarity Test Results")
```

KPSS Stationarity Test Results

kpss_stat	kpss_pvalue
1.695886	0.01

Conclusion

The KPSS test resulted in a p-value of 0.01, significantly below the 0.05 threshold. This statistically confirms that the raw data is non-stationary. This finding validates our use of the ARIMA model, as it correctly applied differencing to stabilize the mean before generating the 2014 projections.